Kodigo

Formamos talento tech en El Salvador y Latinoamérica

AI FOR DEVS JR. 5

ACTIVIDAD SEMANA 3:

IA COMO HERRAMIENTA, NO COMO AUTOR: DESARROLLO GUIADO Y VALIDADO DE UN SCRIPT EN PYTHON

INSTRUCTOR:

ING. RONALD MEJIA

ESTUDIANTE:

ROBERTO CARLOS FLORES QUINTANILLA

🐍 DEMO INTERACTIVA EN LÍNEA • PYTHON 3.12 + PYODIDE

▶ CLIC AQUÍ PARA PROBAR: ANALIZADOR DE LOGS

https://robert-flo.github.io/ia-herramienta-analizador-logs/

Botones de prueba rápida en 1 clic • Editor en vivo • Sin descargas

8 DE NOVIEMBRE DE 2026

1. RESUMEN EJECUTIVO, ENLACES Y DECISIONES TÉCNICAS

El presente informe documenta el desarrollo y validación de un **Analizador de Archivos de Logs en Python** construido bajo la premisa de *IA como Herramienta, no como Autor*. El script lee archivos de eventos (`.log` / `.txt`), identifica el nivel de severidad (`INFO` , `WARNING` , `ERROR`), valida si la línea posee fecha real en formato `YYYY-MM-DD` y contabiliza las líneas mal formateadas o incompletas. Toda la lógica vive en `analizador.py` y se ejecuta de forma idéntica en consola (`main.py`) y en el navegador mediante **Pyodide**.



Tabla 1.1. Fuentes técnicas, repositorios y artefactos para auditoría académica:

Recurso / Entregable	Descripción del Contenido	Enlace Permanente (URI)
Aplicación Web Interactiva GitHub Pages + Pyodide	Ejecuta el módulo canónico `analizador.py` en el navegador con 3 botones de demo en 1 clic y editor en vivo.	robert-flo.github.io/ia-herramienta-analizador-logs/
Repositorio de Código Fuente GitHub (Python 3.12)	Scripts `analizador.py` , `main.py` , `demo.sh` , suite `tests/test_e2e.py` , reflexión y evidencias.	github.com/robert-flo/ia-herramienta-analizador-logs
Archivos de Prueba (.log) Directorio `pruebas/`	Casos buenos (`bien_formado.log`), con errores (`con_errores.log`) y ejemplo de la guía (`ejemplo_enunciado.log`).	github.com/robert-flo/.../pruebas

Decisiones Explícitas Tomadas por el Estudiante (Control Humano)

- **Cómo detectar una línea mal formateada:** Se definió el contrato estricto `[NIVEL] YYYY-MM-DD mensaje` . Cualquier línea sin corchetes, en minúsculas (`[error]`), con nivel fuera de rango (`[DEBUG]`), sin mensaje o con fecha inválida se marca como malformada informando número de línea y causas.
- **Qué hacer cuando falta información (Contabilización dual):** Si una línea tiene nivel válido pero carece de fecha (ej. `[WARNING] Invalid password attempt`), decidí **sumarla al contador de su nivel (`WARNING`) y a la vez contabilizarla como malformada/incompleta** para no ocultar la alerta de seguridad ni el defecto de formato.
- **Cómo estructurar el código:** Separación modular entre lógica pura de validación (`analizador.py`) y manejo de argumentos/consola (`main.py`).
- **Casos borde manejados vs. fuera de alcance:** Líneas en blanco ignoradas del total de eventos (contadas aparte), validación de años bisiestos reales (`2024-02-29` válido vs. `2023-02-29` inválido), soporte BOM UTF-8 (`utf-8-sig`) y captura limpia de errores de archivo. Horas y eventos multilínea quedan fuera de alcance.

Delimitación de Tareas: Apoyo de IA vs. Autoría del Desarrollador

Módulo / Tarea	Delegado a la IA (Herramienta de Apoyo)	Bajo Control del Desarrollador (Estudiante)
Expresiones Regulares	Sintaxis de patrones `re.compile` y grupos de captura en Python.	Definir qué tokens son obligatorios y exigir mayúsculas exactas en `[INFO]` , `[WARNING]` , `[ERROR]` .
Validación de Fechas	Propuesta inicial con `datetime.strptime` (descartada como único filtro).	Detectar el fallo de ancho en `strptime` e imponer validación en 2 capas: regex `4-2-2` + `datetime.date` .
Pruebas y Conteos	Estructura de `unittest` con `subprocess` y carga de Pyodide en HTML.	Diseñar los archivos `.log` y calcular a mano los resultados esperados en `pruebas/esperados.md` .

2. REFLEXIÓN TÉCNICA: DETECCIÓN Y CORRECCIÓN DE IA

Uno de los objetivos centrales de esta actividad es ejercer criterio técnico frente a las sugerencias de la Inteligencia Artificial: no aceptar un bloque de código solo porque ejecuta sin errores sintácticos, sino someterlo a pruebas adversas para comprobar si cumple estrictamente las reglas definidas.

2.1. Ejemplo Concreto: Sugerencia Incorrecta/Incompleta de IA y Corrección Humana

i. Qué sugirió inicialmente la IA (Incompleto para el formato YYYY-MM-DD)

Al consultar cómo validar que la fecha de cada evento cumpliera con el formato `YYYY-MM-DD`, la IA propuso utilizar directamente `datetime.strptime(token, "%Y-%m-%d")` dentro de un bloque `try / except ValueError`:

```
# Sugerencia inicial apoyada por IA:
def fecha_valida(token: str) → bool:
    try:
        datetime.strptime(token, "%Y-%m-%d")
        return True
    except ValueError:
        return False
```

ii. Cómo detecté el problema mediante pruebas de casos borde

Aunque `strptime("%Y-%m-%d")` rechaza fechas inexistentes como `2023-02-29` (año no bisiesto), diseñé pruebas con meses y días de un solo dígito (sin cero a la izquierda). Al probarlo en Python 3.12 (`evidencia/experimento_strptime.txt`), detecté un **falso positivo crítico**:

```
CPython 3.12.3 — Experimento sobre datetime.strptime(valor, "%Y-%m-%d"):
  ACEPTA  '2024-1-2'    → 2024-01-02   [FALSO POSITIVO: mes y día de 1 solo dígito]
  ACEPTA  '2024-1-02'   → 2024-01-02   [FALSO POSITIVO: mes sin relleno de cero]
  ACEPTA  '2024-01-2'   → 2024-01-02   [FALSO POSITIVO: día sin relleno de cero]
  RECHAZA '2023-02-29'  → ValueError: day is out of range for month
```

Confiar únicamente en la sugerencia de la IA habría aceptado como válidas líneas mal formateadas como `[INFO] 2024-1-02 ...`, violando el ancho estricto de 10 caracteres `YYYY-MM-DD`.

iii. Cómo lo corregí en el código final (analizador.py)

Reemplacé la validación única por un filtro de **dos capas explícitas** bajo mi control:

1. **Capa 1 (Ancho exacto ASCII 4-2-2):** Exigir primero `_RE_FECHA_EXACTA = re.compile(r"^\d{4}-\d{2}-\d{2}$")`. Si un token parece fecha pero no tiene ancho `4-2-2`, se reporta: *"fecha con formato incorrecto (se exige YYYY-MM-DD con dígitos ASCII de ancho exacto 4-2-2)"*.
2. **Capa 2 (Calendario gregoriano real):** En `_es_calendario_real(fecha)` se extraen `anio`, `mes` y `dia` y se instancia `datetime.date(anio, mes, dia)` para verificar meses y años bisiestos reales:

```
def _es_calendario_real(fecha: str) → bool:
    try:
        anio, mes, dia = int(fecha[0:4]), int(fecha[5:7]), int(fecha[8:10])
        datetime.date(anio, mes, dia)
    except ValueError:
        return False
    return True
```

2.2. Qué Aprendí sobre el Uso de la IA como Herramienta y no como Reemplazo

- **Saber para qué sirve un bloque de código implica probar sus límites:** La IA sugiere soluciones basadas en patrones frecuentes (como `strptime`), pero corresponde al desarrollador verificar con datos adversos si la función es permisiva en casos borde.
- **Las reglas de validación nunca deben delegarse:** Decidir cómo tratar `[WARNING] Invalid password attempt` (contar la severidad reconocida y a la vez reportar la línea como incompleta) o cómo manejar líneas vacías es una decisión humana de diseño.
- **Definir resultados esperados a mano antes de probar evita el sesgo de confirmación:** Calcular los conteos en `pruebas/esperados.md` antes de ejecutar el script permitió validar el programa contra nuestro criterio técnico.

3. EVIDENCIA DE PRUEBAS Y VERIFICACIÓN DE FUNCIONAMIENTO

Se construyeron tres archivos de prueba en `pruebas/` para evaluar tanto registros bien formados como múltiples errores e inconsistencias, junto con una suite de **8 pruebas automatizadas E2E** (`tests/test_e2e.py`) sobre el CLI real.

Tabla 3.1. Comparativa de Resultados Esperados (calculados a mano) vs. Resultados Obtenidos:

Archivo de Prueba	Escenarios y Casos Evaluados	Total	INFO	WARN	ERR	Mal.	Blc.	Estado
`pruebas/bien_formado.log`	6 registros válidos con fechas reales y mensajes completos.	**6 / 6**	3	2	1	0	0	OK
`pruebas/con_errores.log`	Sin fecha, `2023-02-29` (no bisiesto), minúsculas, `[DEBUG]`, sin corchetes, sin mensaje, 2 blancos y `2024-1-02`.	**8 / 8**	3	1	1	7	2	OK
`pruebas/ejemplo_enunciado.log`	Las 3 líneas exactas de ejemplo del enunciado de la actividad.	**3 / 3**	1	1	1	1	0	OK

3.1. Salida Real en Consola y Suite Automatizada (8/8 Pruebas en OK)

```
$ python3 main.py pruebas/con_errores.log
Origen: pruebas/con_errores.log  |  Líneas en blanco ignoradas: 2
Eventos totales: 8   |   INFO: 3   |   WARNING: 1   |   ERROR: 1   |   Eventos malformados/incompletos: 7
Detalle de malformados (una cuenta por línea, aunque haya varias razones):
  - línea 1: fecha ausente
  - línea 2: fecha no es una fecha de calendario real
  - línea 3: severidad ausente o no reconocida (se exige [INFO], [WARNING] o [ERROR] en mayúsculas exactas)
  - línea 4: severidad ausente o no reconocida (se exige [INFO], [WARNING] o [ERROR] en mayúsculas exactas)
  - línea 5: severidad ausente o no reconocida (se exige [INFO], [WARNING] o [ERROR] en mayúsculas exactas)
  - línea 6: mensaje vacío o ausente
  - línea 10: fecha con formato incorrecto (se exige YYYY-MM-DD con dígitos ASCII de ancho exacto 4-2-2)

$ python3 -m unittest tests.test_e2e -v
Ran 8 tests in 0.268s — OK (bisiesto 2024-02-29, no bisiesto 2023-02-29, ancho 4-2-2, interactivo, vacío,
inexistente)
```

3.2. Evidencia Visual de Paridad en la Interfaz Web (Pyodide)



Caso 1 (`bien_formado.log`): 6 eventos totales, 3 INFO, 2 WARNING, 1 ERROR y 0 malformados.



Caso 2 (`con_errores.log`): 8 eventos totales, 7 malformados desglosados por línea y 2 blancos ignorados.

🐍 **Demo interactiva y archivos de prueba en vivo:** ▶ **PROBAR AHORA: robert-flo.github.io/ia-herramienta-analizador-logs/**